Software Requirements Specification

for

Movie Recommendation System

Prepared by Ibrahim Awan, Jacky Zheng, Omar AlShantout, and Hridi Paul

October 31, 2025

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Table of Contents

Table of Contents **1**
Revision History **1**
1. Introduction **2**
1.1 Purpose 2
1.2 Document Conventions 2
1.3 Intended Audience and Reading Suggestions 2
1.4 Product Scope 2
1.5 References 3
2. Overall Description **3**
2.1 Product Perspective 3
2.2 Product Functions 3
2.3 User Classes and Characteristics 4
2.4 Operating Environment 4
2.5 Design and Implementation Constraints 4
2.6 User Documentation 4
2.7 Assumptions and Dependencies 5
3. External Interface Requirements **5**
3.1 User Interfaces 5
3.2 Hardware Interfaces 6
3.3 Software Interfaces 6
3.4 Communications Interfaces 7
4. System Features **7**
4.1 Data File Parsing 7
4.2 User Rating Management 8
4.3 Recommendation Generation 8
4.4 Use Case Diagram 9
5. Other Nonfunctional Requirements **10**
5.1 Performance Requirements 10
5.2 Safety Requirements 10
5.3 Security Requirements 10
5.4 Software Quality Attributes 11
5.5 Business Rules 11

Revision History

Name	Date	Reason For Changes	Version
Ibrahim Awan	Nov 15, 2025	Original Version	1
Ibrahim Awan	Nov 16, 2025	Add requirement IDs for nonfunctional requirements	1.1

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

1. Introduction

1.1 Purpose

This Software Requirements Specification defines the requirements for a movie recommendation system. The document outlines the complete functionality, scope, and constraints of the system, which is designed to load movie data from text files, allow users to rate films, and allow users to prompt the system for movie recommendations based on historical user ratings. This specification describes the requirements for the entire software system, rather than specific subsystems.

1.2 Document Conventions

This document follows the format and structure outlined in the COMPSCI 2ME3 Milestone #1 template provided on Avenue. Priority levels such as High, Medium, and Low indicate how important each functional requirement is for the software system. All functional requirements are listed individually with clear numbering so that progress and testing can be traced throughout the development process. External interface and nonfunctional requirements are also stated in this document.

1.3 Intended Audience and Reading Suggestions

This document is intended for developers, testers, project managers, and course instructors involved in the design, evaluation, or use of the Movie Recommendation System. Developers and testers should focus on Sections 2 and 3 to understand the functional behavior and data handling requirements, while instructors may reference the document as evidence of proper adherence to the Waterfall methodology. Readers new to the project are encouraged to begin with Sections 1 and 2 for background and context, then proceed to Sections 3-5 for detailed requirements and use cases/interactions. The document is organized to be accessible to both technical and non-technical audiences, ensuring consistent understanding across all stakeholders.

1.4 Product Scope

The Movie Recommendation System is a Java-based desktop application that provides personalized film suggestions by analyzing the genres of movies a user has rated highly. The system allows users to browse available titles, assign ratings from 1 to 5 (inclusive), and receive recommendations for similar films. Its primary educational goal is to help students gain hands-on experience applying the Waterfall software development process, emphasizing structured planning, clear documentation, and systematic implementation. While the system demonstrates core features of a recommendation

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

engine, its larger purpose is to teach good software engineering practices, including modular design, traceability, and maintainable code.

1.5 References

This document references IEEE Std 830-1998 (*IEEE Recommended Practice for Software Requirements Specifications*) for structure and terminology. Additional references include the official course project brief *Milestone #1 – Waterfall Process*, authored by Irene Ye Yuan (McMaster University, 2025), the Oracle Java SE 17 documentation available at https://docs.oracle.com/en/java/javase/17, and relevant Java I/O API references for classes such as *BufferedReader*, *FileReader*, *BufferedWriter*, and *FileWriter*. These sources guide the project's implementation standards and ensure compliance with course and industry best practices.

2. Overall Description

2.1 Product Perspective

The Movie Recommendation System is a stand-alone Java application developed specifically for this academic project. It is not derived from any existing product but serves as the foundational release upon which future versions will be built. The system operates entirely offline, relying on two local text files – movies.txt for the master catalog and my_ratings.txt for user-specific data – to store and retrieve information. It has a modular architecture that separates responsibilities into distinct components, including the file handler, rating manager, and recommendation engine. Each component interacts through clearly defined interfaces.

2.2 Product Functions

At its core, the system performs three essential functions. First, it reads and parses CSV-formatted text files containing movie data and user ratings, handling errors gracefully to maintain data integrity. Second, it enables users to rate movies on a 1-to-5 scale and update existing ratings, persisting these changes to the my_ratings.txt file. Third, it provides personalized recommendations by analyzing the genres of highly user-rated movies and ranking unwatched films based on similarity. Additional capabilities, such as searching or filtering by genre, may be incorporated in future releases depending on developer resources and delivery deadlines. Please see Section 4 for a much more detailed description of system functionality.

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

2.3 User Classes and Characteristics

The primary users of this system are general movie enthusiasts who wish to receive personalized recommendations based on their own ratings. They are expected to have basic computer literacy and the ability to run Java programs through an IDE or command-line interface. No prior technical or programming knowledge is required. A secondary user group includes student developers and teaching assistants responsible for developing, testing, and evaluating the software. The system does not incorporate authentication or user-level permissions, as it is designed for individual, local, optionally offline, usage. Notice that the application is meant for a single user interacting with the application (i.e. the user's machine is the server).

2.4 Operating Environment

The application runs on any desktop or laptop system capable of executing Java SE 17 or later. It is compatible with Windows 10/11, macOS, and major Linux distributions that support a Java runtime environment (JRE) of version 17 or later. The system requires minimal hardware resources and functions entirely offline, reading and writing from local files. No network connectivity or external dependencies are required, ensuring portability and simplicity of setup.

2.5 Design and Implementation Constraints

The software must be implemented exclusively in Java using standard libraries, without third-party dependencies or external frameworks. All file operations may utilize the classes *BufferedReader*, *FileReader*, *BufferedWriter*, and *FileWriter*. The program must adhere to consistent coding conventions, include meaningful inline documentation, and maintain a clear separation between data management, business logic, and user interaction layers. Development and version control must be managed through GitHub Classroom, with meaningful commit histories that demonstrate progressive implementation under the Waterfall model. These constraints ensure maintainability, academic integrity, and technical uniformity across all project submissions.

2.6 User Documentation

A comprehensive README.md file accompanies the software, detailing installation instructions, compilation and execution commands, directory structure, and Java version requirements. This documentation serves as the user's primary reference for running and understanding the system. Additionally, the project deliverables include an Informal Testing Report that outlines test setup, expected output, and actual output. The program provides sufficient instructions via terminal output on startup; no additional user documentation is required at this point. However, documentation may be added in the form of a user manual as more features are delivered over time.

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

2.7 Assumptions and Dependencies

It is assumed that a properly formatted movies.txt file is available prior to running the program. The application depends on the Java runtime environment being correctly installed and accessible from the system path. The system assumes that it has permission to read and write files within its working directory. If the my_ratings.txt file does not exist, the system will automatically create it when the user submits their first rating. The project relies on version control through GitHub Classroom for collaborative development. Any future changes to data formats, Java versions, or execution environments may require code adjustments to preserve compatibility.

3. External Interface Requirements

The requirements listed in this section are not prefixed with a unique identifier because they will not be referenced in the testing documentation. The testing documentation involves cases that assess functional requirements whereas these requirements should be part of a E2E automation testing framework that deploys the software in environments with various operating systems, free disk space, etc. to ensure system reliability and robustness.

3.1 User Interfaces

3.1.1 UI Models

The program will use a system-provided command-line interface (CLI) to render outputs and receive inputs. All text will be UTF-8 and will render correctly on a 80 x 24 terminal without wrapping.

3.1.2 Flow

Upon start, the program will display a title banner and a numbered main menu with all the available actions. There will be a prompt symbol '>', which will appear after each screen requiring user input. After any action is completed, the system will return to the main menu.

3.1.3 Commands

The following options will be included on the menu:

1) Rate a Movie
2) Display Movie Catalog
3) Generate Movie Recommendations
q) Quit Application

3.1.4 Input Validation and Errors

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

If invalid input is entered, the program will not terminate. The program will instead handle the error with a single-line message which will re prompt the user to enter valid inputs. All error messages will all begin with "*ERROR:"* and proceed by stating the cause and corrections (e.g: *ERROR: Rating must be an integer 1-5).* Success messages will all begin with "OK: " (e.g: *OK: Rating Saved.)*

3.2 Hardware Interfaces

- Runs on any desktop/laptop which contains a local file system directly accessible by the user.
- Program will require at least 1 MB free disk space for ratings/logs.
- Program will require direct read access to the project directory and write access to create/update user files.
- Program will use user-native system clock to timestamp ratings with the date only.

3.3 Software Interfaces

3.3.1 Runtime and OS

The program will require JRE (e.g. 17+) on the deployed computer.

3.3.2 Standard Libraries Only

File input/output will be handled using Java standard libraries, (e.g java.io.*, java.nio.file.*). The date will be taken using java.time.*. There will be no use of external libraries.

3.3.3 Local Data Files

Input catalog file: movies.txt (CSV): fields per row: *movie_id, director, year, genres, avg_rating, num_ratings,* where genres are separated by semi-colons (e.g *Action;Sci-Fi)*.

User data file: my_ratings.txt (CSV): fields per row: *movie_id, rating, date* where rating can be between 1-5, where date = YYYY-MM-DD.

If my_ratings.txt does not exist upon program startup, the program will create it.

3.3.4 Inter-module contracts

The UI will pass user input to the business logic (BI) layer; the BI layer will throw exceptions back to the UI layer for the UI to print error messages to the terminal. If the BI layer completes executing logic without encountering an error, the UI will print a success message to the console.

Shared Structure:

Movie *{ id:int, title:String, director:String, year:int, genres:List<Genre >, averageRating:double, numberOfRatings:int }*

The recommendation service consumes the set of movies rated >= 4 and returns a list of Movie objects sorted in descending similarity scores.

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

3.3.5 Parsing

When parsing the CSV, whitespace will be automatically trimmed, and field counts/types are validated. All file operations will have *IOException* handled which will report *ERROR:* with prompts to guide the user action.

3.4 Communications Interfaces

- This application will be completely offline; there will be no network communication. There will be no attempt from this application to attempt outbound collections.

4. System Features

4.1 Data File Parsing

4.1.1 Description and Priority

The system must be able to successfully read and parse both data files (movies.txt and my_ratings.txt). This feature lays the groundwork for implementation of subsequent features as it is responsible for loading data from the data files into java objects that are utilized by the recommendation algorithm. Therefore, this feature is of High priority. Also note that this is more of an "intermediate" feature that is a pre-requisite to delivering the following two customer-facing system features.

4.1.2 Stimulus/Response Sequences

N/A. The sequence of user actions that trigger data file parsing are covered in sections 4.2.2 and 4.3.2.

4.1.3 Functional Requirements

REQ-1: The system must be able to read each record in the movies.txt file into a java object. These objects, when aggregated into a list, represent the movie master catalog that can be used by the recommendation system.

REQ-2: The system must be able to read each record in the my_ratings.txt file into a key-value pair mapping a movie id to a user rating. These kv pairs, when aggregated, represent the movies that have been rated by the user along with their ratings.

REQ-3: The system must be able to gracefully handle parsing errors encountered when parsing movies.txt or my_ratings.txt. If an error is encountered, the system must log the error to the console and continue with valid data. Refer to Section 3.3.3 to see the format of movie data in movies.txt.

REQ-4: If movies.txt does not exist or is empty, the system must log a message to the console and exit gracefully since the system cannot function without movie data.

REQ-5: If my_ratings.txt does not exist, the system must treat this as normal for a new user and create the file when the user rates their first movie.

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

REQ-6: Generalizing REQ's 3-5, the system must never crash due to missing files or malformed data.

REQ-7: The system must allow the user to display the movie catalog in the terminal so that users can view which movies are available for them to rate. This requirement references command 2 in Section 3.1.3.

4.2 User Rating Management

4.2.1 Description and Priority

The system must be able to allow users to rate movies with persistence to my_ratings.txt. Furthermore, users must have the ability to update existing ratings with an updated timestamp. This feature is of High importance.

4.2.2 Stimulus/Response Sequences

The user will have a "Rate a Movie" selection available to them. If the user selects this option, the user will then be prompted to provide the relevant movie id that they would like to rate as well as an associated rating. The system will check if the provided movie id is valid (i.e. a movie with a matching id exists within movies.txt) and that the associated rating is valid (i.e. rating between 1-5 inclusive). The system will then check if the user has already rated this movie in the past. If so, the system will simply update the existing rating with the provided rating. Otherwise, the system will append a new row in my_ratings.txt with information about the movie id, rating, and timestamp.

4.2.3 Functional Requirements

REQ-8: The system must be able to allow users to rate movies with persistence to my_ratings.txt, including the ability to update existing ratings.

REQ-9: The system must be able to handle a cold start (i.e. when my_ratings.txt does not exist). In this case, it should create a new file when the user rates their first movie.

REQ-10: The system must prevent users from rating a movie that does not exist in the master catalog. In other words, if the movie id provided by the user does not match any movie id present in movies.txt, the system should not modify my_ratings.txt. Instead, it should notify the user that they are attempting to rate an invalid movie and re-prompt the user for a movie id. The system must not crash if the user provides an invalid movie id.

REQ-11: The system must prevent users from assigning an invalid rating to a movie. The system must verify that the user-provided rating is an integer between 1 and 5. If this is not the case, the system should re-prompt the user to enter a valid rating (or exit to the main menu). If and only if the user enters a valid rating will the system allow writing to my_ratings.txt file.

4.3 Recommendation Generation

4.3.1 Description and Priority

The system must be able to generate movie recommendations for the user by analyzing user rating data. That is, the system must be able to analyze the content in

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

my_ratings.txt and provide the user with unwatched movie recommendations from the master catalog contained within movies.txt. This feature is of High importance.

4.3.2 Stimulus/Response Sequences

The user will have a "Generate Movie Recommendations" selection available to them. If the user selects this option, the recommendation algorithm will be triggered, and the user will be provided with an ordered list of top 5 movie recommendations.

4.3.3 Functional Requirements

REQ-12: The system must be able to generate movie recommendations using user ratings stored within my_ratings.txt. The algorithm exercised by the recommendation engine should assign similarity scores to each unwatched movie in movies.txt based on user ratings and the top 5 movies with the highest similarity scores will then be displayed to the user.

REQ-13: The system must be able to generate movie recommendations if there are no user ratings yet by considering the average rating score and number of ratings of all movies within the master catalog.

4.4 Use Case Diagram

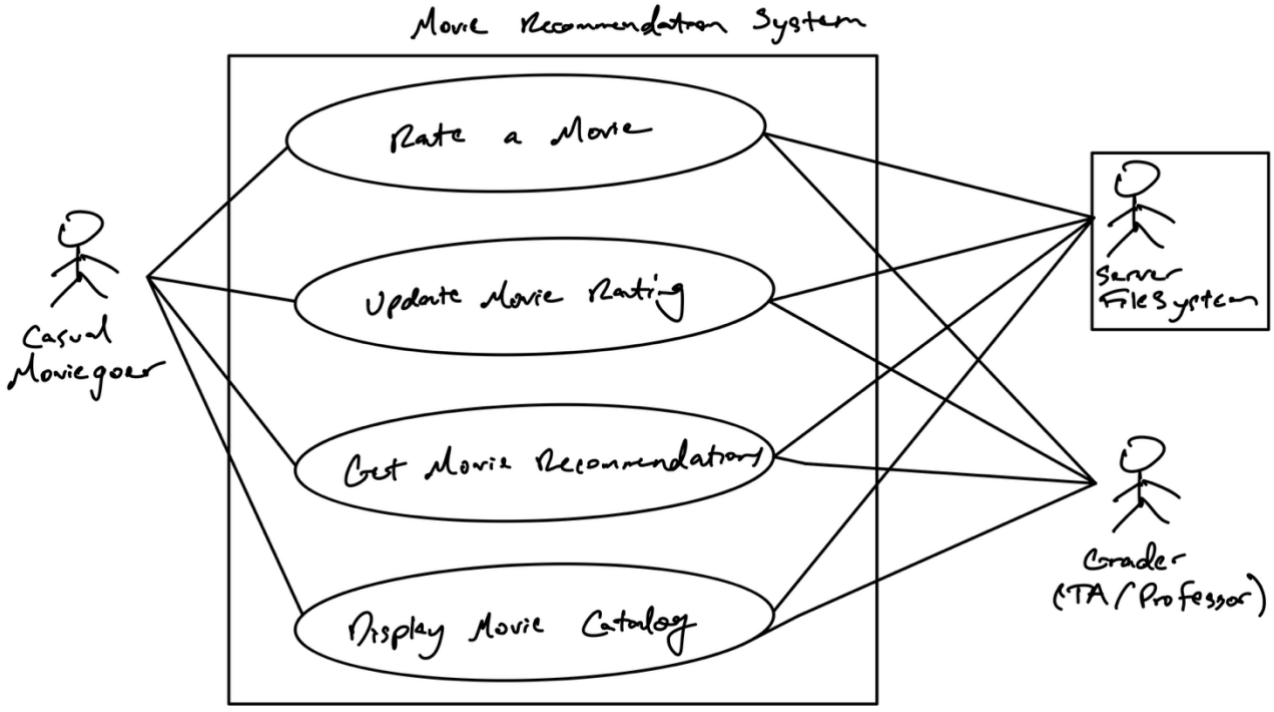


Figure 1: Use Case Diagram of the Movie Recommendation System

The use case diagram for the movie recommendation system is shown above. The User Rating Management system feature (Section 4.2) corresponds to both "Rate a Movie" and "Update Movie Rating" use cases and the Recommendation Generation feature (Section 4.3) corresponds to the

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

"Get Movie Recommendations" use case. The last use case outlines another way in which a user can interact with the system, and its purpose is to provide a convenient way for the user to view the master movie catalog; without this option, the user would be required to manually open the movies.txt file from the file system.

Primary and secondary human actors are also shown in the diagram above. The "Grader" is shown as secondary actor because he/she does not utilize the movie recommendation system for its "theoretical" intended purpose. The Grader interacts with the system to assess the developed product against the pre-defined project rubric. There was also some discussion to include a secondary "Student Developer" actor but by that logic, any software product would have a secondary actor that represents the developer interacting with the system to develop and test the software. Therefore, it was decided to remove the Student Developer actor from the diagram.

The server file system that stores movies.txt and my_ratings.txt is shown as a secondary, system actor. Because the java program reads/writes to the text files that are stored on the file system and write operations to my_ratings.txt are persistent, it makes logical sense to add the file system itself as a secondary, system actor. It is important to note that although movies.txt and my_ratings.txt are stored on the file system, they are part of the application itself.

5. Other Nonfunctional Requirements

Non-functional requirements are listed below with the "NFR" prefix

5.1 Performance Requirements

NFR-1: The system must be able to load and parse the movies.txt file containing at least 1,000 movie entries within 1 second on a standard desktop computer running Java 17 or higher.
NFR-2: The program must be able to print the movie catalog to the terminal and complete the movie rate operation within 0.5 seconds.
NFR-3: The recommendation algorithm should generate a recommendation list within 1 second using content-based filtering based on genre similarity.
(Profiling tools may be utilized to ensure performance requirements are satisfied)

5.2 Safety Requirements

NFR-4: The application should not modify or delete files outside the designated /data directory. Doing so may cause loss of sensitive information.
NFR-5: The application must not corrupt movies.txt/my_ratings.txt when executing read/write operations.
NFR-6: The application must ensure all I/O resources are always closed upon completion of read/write operations by utilizing the built-in java try-with-resources feature

5.3 Security Requirements

NFR-7: The program should restrict access to external file paths—it must only read from and write to the designated /data directory.
NFR-8Error messages should clearly describe the issue to the user in the terminal without exposing system-level or private user information.
NFR-9: User data in my_ratings.txt must be private and inaccessible to other users; no external network connections or data transmissions are permitted.

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

5.4 Software Quality Attributes

Quality Attribute	Description
Usability	Simple command-line interface with numbered menu options for rating and recommendations. Error messages are clear and concise.
Maintainability	Modular Java classes (Movie, RatingManager, RecommendationEngine, FileHandler) enable easy updates and debugging. Code follows Java naming conventions and includes inline documentation.
Reliability	The system continues to operate with partial or malformed input files and logs all errors to the console.
Portability	The system can run on any platform that supports JRE 17+ without requiring additional libraries.
Scalability	The design supports adding more complex recommendation algorithms without modifying core data structures.
Testability	Clear separation of concerns between file parsing, business logic, and I/O allows each component to be unit tested independently.

5.5 Business Rules

- Only valid movie IDs listed in movies.txt can be rated by the user.
- Ratings must be integers between 1 and 5 inclusive.
- When a user re-rates a movie, the latest rating and rating timestamp overwrite the old data.
- Recommendation generation is based solely on genre overlap and user ratings (≥4 stars); if no ratings exist, popular movies (based on avg_rating and num_ratings) are displayed instead.
- Data files must maintain their specified CSV formats exactly as defined in the project description.

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.